

INT315

CLUSTER COMPUTING

Prerequisites & Foundations Handbook

Before Unit I: Building the Foundation

Module 1 — Core Linux & Shell Essentials | Module 2 — Distributed
Systems Fundamentals | Module 3 — The Big Data Evolution

Session 2026-27

Companion Study Material — prepared for course mates of INT315

Table of Contents

How To Use This Handbook.....	4
MODULE 1 — Core Linux & Shell Essentials.....	5
1.1 File System Navigation.....	5
1.2 Environment Variables — PATH, JAVA_HOME, SPARK_HOME	6
1.3 Basic Networking for Clusters — SSH, Ports, IP Bindings.....	8
1.4 Memory Management — JVM, Heap vs. Off-Heap, Garbage Collection.....	10
1.5 File Permissions & Ownership	12
1.6 Process Management — Seeing and Controlling Running Services	13
1.7 Shell Scripting Basics — Automating Cluster Tasks	14
1.8 Archives & Compression — tar, gzip, and Why Big Data Cares	15
MODULE 2 — Distributed Systems Fundamentals.....	17
2.1 What Is Distributed Computing? Vertical vs. Horizontal Scaling	17
2.2 The Bottleneck Problem — Network I/O vs. Disk I/O vs. RAM Speed	18
2.3 Cluster Architectures — Master/Worker Topology & Resource Managers	19
2.4 Fault Tolerance, Replication, and a Gentle Intro to the CAP Theorem.....	21
2.5 Data Partitioning & Parallelism.....	22
2.6 Serialization — Getting Objects Across the Network	22
MODULE 3 — The Big Data Evolution	24
3.1 Why Traditional Single-Node Databases Fail at Scale.....	24
3.2 HDFS Refresher — NameNode & DataNode	25
3.3 MapReduce Execution Flow Refresher	25
3.4 A Short Timeline — How We Got From Hadoop to Spark to Kafka	27
3.5 A Quick Tour of the Hadoop Ecosystem	28
3.6 Batch vs. Real-Time Analytics — A Preview of Unit I's Opening Line	29
MODULE 4 — Career Opportunities & Industry Exposure.....	30
4.1 Job Roles This Course Feeds Into.....	30
4.2 Where This Actually Gets Used — Industry Scenarios.....	30
4.3 Certifications & Learning Paths Worth Knowing About.....	31
4.4 A Realistic Skill Roadmap After INT315.....	31
MODULE 5 — Interview Preparation: 50 Q&A on Prerequisites & Foundations	33
A. Linux, Shell & Environment (Q1–Q12).....	33
B. Distributed Systems Fundamentals (Q13–Q28)	35
C. Big Data Evolution, HDFS, MapReduce & Ecosystem (Q29–Q50).....	38

You're Ready for Unit I	42
--------------------------------------	-----------

How To Use This Handbook

Cluster Computing (INT315) assumes you already have comfort with a few layers that most students never got taught explicitly: the Linux command line, how a JVM manages memory, what actually happens when many computers talk to each other over a network, and why Hadoop existed before Spark showed up to fix it. This handbook fills exactly that gap — nothing more, nothing less — so that when Unit I opens with "limitations of MapReduce in Hadoop," you already know what MapReduce is, why it is slow, and what a cluster even looks like physically and logically.

Every topic below follows the same five-beat rhythm you'll see throughout the whole course:

- Theory & Mechanics — the plain-English explanation, then what's really happening under the hood.
- A Story / Comic Metaphor — a colorful scenario that makes the abstract idea sticky.
- Industry Context — why production systems care about this at all.
- Hands-on Practice — real terminal commands and exercises you can run right now.
- Self-Assessment — a few questions plus one challenge to prove you've got it.

How to work through this

Read a topic, then immediately open a terminal (or a free cloud shell / VM) and run the commands yourself. Muscle memory beats re-reading every time.

Don't rush Module 1. Ninety percent of "Spark isn't working" bugs in this course trace back to a misunderstood environment variable, a wrong Java version, or a port that's blocked — all Module 1 topics.

MODULE 1 — Core Linux & Shell Essentials

Spark, Hadoop, and Kafka all live and breathe on Linux. Every install guide, every cluster script, every log file you will touch in this course assumes you can navigate a filesystem, read and set environment variables, understand basic networking, and reason about how the JVM uses memory. This module builds that muscle.

1.1 File System Navigation

Linux organizes everything — files, devices, even running processes — into a single tree rooted at `/`. There is no `"C:\"` here; every drive, USB stick, or network mount gets grafted onto this one tree at some directory. As a cluster computing student, you'll live mostly in a handful of these directories:

- `/home/<user>` — your personal space; this is where you'll typically download and unzip Spark.
- `/opt` or `/usr/local` — the conventional home for manually installed software like spark, hadoop, kafka.
- `/etc` — system-wide configuration files (network settings, hostnames, sometimes Java's system config).
- `/var/log` — where services write their log files; when a Spark job fails mysteriously, this is often where the real error message hides.
- `/tmp` — scratch space; Spark itself uses `/tmp` heavily for shuffle spill files unless reconfigured.

Core navigation commands

```
# Where am I right now?
pwd

# List contents of the current directory (long format, human-readable sizes,
show hidden files)
ls -lah

# Move into a directory
cd ~/spark-lab

# Move up one level, then back to home
cd ..
cd ~

# Create a directory tree in one shot
mkdir -p ~/spark-lab/data/raw

# Copy, move, remove
cp input.csv ~/spark-lab/data/raw/
mv old_name.csv new_name.csv
rm -rf ~/spark-lab/data/raw/tmp_files
```

```
# Peek inside a file without opening an editor
head -n 20 access.log
tail -f spark-worker.log      # -f = follow, live-stream new lines

# Find a file anywhere under a directory
find / -iname 'spark-submit*' 2>/dev/null
```



The Filing Cabinet City

Imagine the Linux filesystem as a city built as one giant filing cabinet, planted at a single root — literally called `/`. Every drawer (directory) can contain more drawers or actual documents (files).

You (the shell) are a courier standing in one drawer at a time — that's your "present working directory," what `pwd` tells you. "`cd`" is you physically walking into another drawer. "`ls`" is you shining a flashlight into the current drawer to see what's inside.

When you later install Spark, you're basically renting a new drawer (say `/opt/spark`) and unpacking a toolbox into it. Every time a command says "file not found," 90% of the time the courier (you) is standing in the wrong drawer.



Practice now

Open a terminal, run: `mkdir -p ~/int315/module1 && cd ~/int315/module1 && pwd` — confirm the path printed matches where you expect to be.

1.2 Environment Variables — `PATH`, `JAVA_HOME`, `SPARK_HOME`

An environment variable is a named value that the shell keeps in memory and hands to every program it launches. Programs use them as configuration without you having to pass a flag every single time. Three of these variables will make or break your Spark installation:

- `PATH` — an ordered, colon-separated list of directories the shell searches, in order, whenever you type a command name without a full path. If Spark's `bin/` folder isn't on `PATH`, typing `spark-shell` gives "command not found" even though the file physically exists.
- `JAVA_HOME` — points to the root folder of your JDK installation (the folder that contains `bin/java`). Spark is written largely in Scala, which compiles to JVM bytecode, so Spark literally cannot start without finding a working JVM at `JAVA_HOME`.
- `SPARK_HOME` — points to the folder where you extracted Spark. Scripts like `spark-submit` and `spark-shell` use this internally to locate Spark's own jars and configuration files, even after you've added its `bin/` to `PATH`.

Setting them up

```
# 1. Check if Java is already installed and see its path
```

```

java -version
which java

# 2. Find the real JDK install location (java is often a symlink chain)
readlink -f $(which java)
# e.g. /usr/lib/jvm/java-17-openjdk-amd64/bin/java -> JAVA_HOME is the folder
ABOVE bin/

# 3. Export the variables for THIS terminal session only
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export SPARK_HOME=/opt/spark
export PATH=$PATH:$JAVA_HOME/bin:$SPARK_HOME/bin:$SPARK_HOME/sbin

# 4. Verify
echo $JAVA_HOME
echo $PATH
spark-shell --version

# 5. Make it PERMANENT by appending the same export lines to your shell's
# startup file, then reloading it
nano ~/.bashrc      # paste the export lines at the bottom, save, exit
source ~/.bashrc    # reload without closing the terminal

```

The Sticky Notes on the Courier's Vest

Picture the shell as that same courier from before, but now wearing a vest covered in sticky notes. Each sticky note is an environment variable.

One note says `JAVA_HOME`: "the java toolbox lives in Drawer 17." Another says `SPARK_HOME`: "the spark toolbox lives in Drawer 42." A third, `PATH`, is a special note listing a whole route of drawers to check, in order, whenever someone shouts a tool's name at the courier without telling them exactly where it is.

When you type `spark-shell`, the courier doesn't magically know where that tool lives — they walk the `PATH` route, drawer by drawer, until they find something named `spark-shell`. If Drawer 42's `bin/` folder was never added to that route, the courier gives up and yells 'command not found,' even though the tool is sitting right there in Drawer 42.

Why production teams obsess over this

Real clusters run dozens of services (Spark, Hadoop, Kafka, Zookeeper, Hive) that all need their own `*_HOME` variables set consistently across every single machine in the cluster. A single worker node with a mismatched `JAVA_HOME` is a classic cause of "works on master, fails on worker" bugs — the kind that costs hours in production. That's exactly why tools like Ansible, Docker, and Kubernetes exist: to guarantee identical environment configuration across hundreds of machines instead of relying on someone manually editing `.bashrc` on each box.

⚠ Common trap

Setting a variable with `export` only affects the CURRENT terminal session and any programs launched from it. Close the terminal and it's gone. Forgetting to also add it to `~/.bashrc` (or `~/.zshrc`) is the #1 reason "it worked yesterday but not today."

🧐 Check yourself — 1.1 & 1.2

Q1. What is the practical difference between running ``cd`` with no arguments and running ``cd ..``?

Q2. Why does Spark specifically need `JAVA_HOME` rather than just relying on `PATH` containing `java`?

Q3. If ``spark-shell`` isn't found after installation, name two different variables that could be misconfigured, and how you'd check each.

Hands-on Challenge:

Install a JDK on your machine (or a free cloud shell), locate its home directory with ``readlink -f $(which java)``, export `JAVA_HOME` manually, then confirm with ``echo $JAVA_HOME`` and ``java -version``. Finally make it permanent in your shell's rc file and prove it survives a fresh terminal window.

1.3 Basic Networking for Clusters — SSH, Ports, IP Bindings

A "cluster" is, physically, nothing more than several separate computers that can talk to each other over a network. Everything from Spark's master-worker communication to Kafka's producer-consumer pipeline rides on top of a handful of networking basics.

SSH — remote control over the network

SSH (Secure Shell) lets you log into and run commands on a remote machine as though you were sitting in front of it, with the traffic encrypted. In a real cluster, the master node uses SSH to start and stop services on every worker node — that's literally how Hadoop's `start-all.sh` and Spark's standalone cluster scripts work internally.

```
# Connect to a remote machine
ssh username@192.168.1.101

# Generate a public/private key pair (press Enter through the prompts for defaults)
ssh-keygen -t rsa -b 4096

# Copy your PUBLIC key to a remote machine so it can log you in without a password
ssh-copy-id username@192.168.1.102

# Now this works with no password prompt -- this is EXACTLY what lets a
# Spark/Hadoop master start services on every worker with one command
ssh username@192.168.1.102 'hostname && uptime'
```


IP addresses and hostname binding

Every machine on a network needs an address. A cluster service must be told which network interface to "bind" to — 0.0.0.0 means "listen on every interface / accept connections from anywhere," while 127.0.0.1 (localhost) means "only accept connections from this same machine." A classic beginner bug is starting Spark's master bound to localhost and then wondering why worker nodes on other machines can't register with it.

```
# See this machine's IP address(es)
ip addr show
hostname -I

# Test whether a remote host is reachable at all
ping -c 4 192.168.1.101

# Test whether a SPECIFIC PORT on a remote host is open and accepting
connections
nc -zv 192.168.1.101 7077      # Spark master's default RPC port
telnet 192.168.1.101 8080    # Spark master's default Web UI port
```

Ports you'll meet constantly in this course

- 22 — SSH (remote login / cluster start-up scripts)
- 7077 — Spark Standalone master's RPC port (where workers register)
- 8080 — Spark master Web UI (view cluster status in a browser)
- 4040 — Spark application UI (per running job — DAGs, stages, tasks)
- 9000 / 8020 — HDFS NameNode RPC port (client / DataNode communication, version dependent)
- 9092 — Kafka broker's default client port



The Walkie-Talkie Convention Center

Think of a cluster as a convention center full of walkie-talkies. Each machine is a person; each PORT is a specific CHANNEL that person is willing to listen on for a specific topic. Port 7077 is "the Spark-master-only channel." Port 9092 is "the Kafka-broker-only channel."

Binding to 127.0.0.1 is like whispering into your walkie-talkie with the volume so low only you can hear it — technically transmitting, but useless for anyone else in the building. Binding to 0.0.0.0 is holding the button and speaking normally so anyone in range on that channel can hear you.

SSH-with-keys is the VIP badge system: once the master has your face on file (your public key), security (the remote machine) waves you through every checkpoint without asking for ID (a password) again — which is exactly how a master node quietly starts services on ten worker nodes in one command.



Why this matters at scale

In real deployments, ports are locked down tightly by firewalls and security groups (AWS Security Groups, GCP firewall rules, on-prem iptables). A huge share of "my Spark cluster won't connect"

incidents in production are firewall misconfigurations, not Spark bugs — the master's port 7077 simply isn't reachable from the worker's subnet. Cloud engineers spend real time drawing network diagrams of exactly which port needs to be open between which node types before a cluster ever runs a single job.



Check yourself — 1.3

Q1. Why would a Spark worker on a different physical machine fail to connect if the master is bound to 127.0.0.1?

Q2. What's the practical benefit of SSH key-based authentication over password authentication for cluster automation scripts?

Q3. Name the default port for the Spark master Web UI and explain what you'd expect to see there.

Hands-on Challenge:

If you have access to two machines (or two VMs / containers) on the same network, set up passwordless SSH between them using `ssh-keygen` and `ssh-copy-id`, then run a single remote command from machine A that prints machine B's hostname and current uptime.

1.4 Memory Management — JVM, Heap vs. Off-Heap, Garbage Collection

Spark is a JVM application (even when you write PySpark — a JVM process still runs underneath, coordinating with a Python process). Nearly every Spark performance tuning conversation eventually becomes a memory conversation, so understanding the JVM's memory model now will save you real pain in Units III and VI.

The JVM memory model, simplified

When a JVM process (like a Spark executor) starts, the operating system hands it a chunk of RAM. The JVM then subdivides that chunk into regions:

- **Heap** — where almost all your objects live (RDD partitions cached in memory, DataFrame rows being processed, ordinary Java/Scala/Python-bridge objects). This is the memory pool that Garbage Collection manages.
- **Stack** — one per thread; holds local variables and method call frames. Small, fast, and not something you tune directly in Spark.
- **Off-heap memory** — memory the JVM allocates OUTSIDE the managed heap, invisible to the garbage collector. Spark deliberately uses off-heap memory (via `spark.memory.offHeap.enabled`) for large buffers because it avoids GC overhead entirely — the tradeoff is you must size and free it manually through configuration.
- **Metaspace / PermGen (legacy)** — stores class metadata itself, not your data.

Garbage Collection (GC), briefly

The heap fills up with objects as your program runs. The Garbage Collector periodically pauses (or partially pauses) the application to identify objects nothing refers to anymore and reclaim that memory. This pause is called a "GC pause," and in Spark, long GC pauses on an executor are one of the most common causes of a job that mysteriously "hangs" for a few seconds at a time — the JVM isn't stuck, it's cleaning house.

```
# See the JVM's default heap sizing behaviour on this machine
java -XX:+PrintFlagsFinal -version | grep -i heapsize

# Launch any JVM app with an explicit heap size (min 512MB, max 2GB here)
java -Xms512m -Xmx2g -jar someapp.jar

# Print live GC activity while a JVM program runs (handy later when tuning Spark executors)
java -Xlog:gc -jar someapp.jar

# Later in the course you'll set this directly for Spark executors, e.g.:
#   spark-submit --executor-memory 4g --conf spark.memory.offHeap.enabled=true \
#   --conf spark.memory.offHeap.size=1g my_job.py
```



The Overflowing Studio Apartment

Picture the JVM Heap as a studio apartment (your "managed" living space). Every time your program creates an object — a variable, a row of data, a list — it's like bringing home a new piece of furniture. The apartment isn't infinite, and eventually it starts to feel cramped.

The Garbage Collector is a very diligent, slightly annoying roommate who periodically walks through the apartment and asks each item of furniture: "is anyone still using you?" If nothing points to it anymore, out it goes. But while the roommate is doing this sweep, you (the running program) sometimes have to just stand still and wait — that's your GC pause.

Off-heap memory is like renting a storage unit down the street instead of cramming everything into the apartment. Nothing in the storage unit gets randomly cleaned out by the roommate — but YOU are now responsible for knowing what's in there and clearing it out yourself, because the diligent roommate never even looks at it.



Why Spark engineers care so much

Under-sizing executor memory triggers constant garbage collection and, in the worst case, `OutOfMemoryError` crashes that kill an entire stage of a job. Over-sizing wastes cluster money — you're paying for RAM that sits idle. Real production tuning is a careful balance of executor-memory, executor-cores, and how much of that memory Spark reserves for its own execution/storage vs. how much it leaves as "user memory." Companies running large Spark clusters on AWS/GCP treat this tuning as a direct line item on their cloud bill.



Check yourself — 1.4

Q1. In one sentence, what triggers a Garbage Collection pause?

Q2. Why might a Spark team deliberately choose to enable off-heap memory even though the GC won't manage it?

Q3. What's the practical symptom you'd observe in a running application if GC pauses become very long or very frequent?

Hands-on Challenge:

Write and run a tiny Java or Scala program that allocates a large array in a loop (e.g. millions of small objects) inside a `while(true)`. Run it with `java -Xmx64m -Xlog:gc YourProgram` and observe the GC log lines scroll by until it eventually throws `OutOfMemoryError` — you're watching, live, the exact failure mode Spark executors avoid through careful memory tuning.

1.5 File Permissions & Ownership

Linux is a multi-user operating system at its core, and clusters take that seriously — the user account a service runs as (often a dedicated `hadoop` or `spark` system user, never root) determines exactly what it can read, write, or execute. Misconfigured permissions are one of the most common "Permission denied" errors you'll hit when setting up HDFS directories or Spark's local scratch space.

Every file and directory carries three permission triplets — owner, group, and others — each made of read (r), write (w), and execute (x):

```
$ ls -l spark-defaults.conf
-rw-r--r-- 1 hadoop hadoop 842 Jan 10 09:14 spark-defaults.conf
  ^^^^^^^^^  ^^^^^  ^^^^^
  perms      owner  group

# Read the 10-character string in three groups of three (skip the leading '-' =
# file type):
#  rw-  r--  r--
#  owner grp other

# Give the owner execute permission on a script
chmod u+x start-cluster.sh

# Set exact permissions numerically (owner=rwx=7, group=rx=5, other=none=0)
chmod 750 start-cluster.sh

# Change ownership -- common right after downloading Spark as root
sudo chown -R hadoop:hadoop /opt/spark

# Check which user you currently are, and switch to a service account
whoami
sudo su - hadoop
```

 Rule of thumb for this course

Never run Spark, Hadoop, or Kafka as root in anything beyond a quick personal test. Create a dedicated user (commonly named `hadoop` or `spark`), own all the installation directories with that user, and run every daemon and job as that user — this is standard practice on every real cluster you'll touch after graduation.

1.6 Process Management — Seeing and Controlling Running Services

A Spark or Hadoop cluster is, underneath everything, just a collection of long-running background processes (daemons): a NameNode process, several DataNode processes, a Spark master process, several worker processes. Being able to see what's running, how much CPU/memory it's consuming, and how to stop it cleanly is a daily skill.

```
# List every running process for the current user, tree-style
ps -ef | grep java

# Live, auto-refreshing view of CPU/memory usage per process
top
htop          # nicer, color version if installed

# Find the exact Process ID (PID) of a specific service, e.g. Spark's master
jps           # JVM-specific process viewer -- shows friendly names like 'Master',
              'Worker', 'NameNode'

# Gracefully stop a process by PID
kill 4821

# Force-kill an unresponsive process (last resort)
kill -9 4821

# Send a background job to the background and check on it later
nohup spark-submit my_job.py > job.log 2>&1 &
jobs
disown
```

 The Backstage Crew

If your terminal commands are the audience-facing performance, daemons are the backstage crew working the whole show without ever stepping into the spotlight -- the NameNode quietly maintaining its card catalog, the Spark master quietly tracking its workers, all running continuously in the background.

`jps` is like walking backstage with a clipboard that already knows everyone's job title -- instead of a wall of cryptic numbers, it tells you plainly "PID 4821 is the Master, PID 4901 is a Worker." `kill` is you politely tapping a crew member on the shoulder and asking them to wrap up (SIGTERM); `kill -9` is pulling the power cord out of the wall because they simply won't stop (SIGKILL) -- and just like unplugging a real

machine mid-task, this should always be your last resort, since it gives the process no chance to clean up open files or in-flight writes.

Check yourself — 1.5 & 1.6

Q1. What do the three permission triplets on a file represent, and in what order do they appear?

Q2. Why is it considered bad practice to run cluster daemons as the root user?

Q3. What's the practical difference between `kill <pid>` and `kill -9 <pid>`, and why would you prefer the former whenever possible?

Hands-on Challenge:

Create a small shell script `myjob.sh` that just sleeps for 60 seconds, make it executable with `chmod`, launch it in the background with `nohup ... &`, find its PID using `jps` or `ps -ef | grep myjob`, and cleanly terminate it with `kill` before the 60 seconds elapse.

1.7 Shell Scripting Basics — Automating Cluster Tasks

Every cluster start/stop script you'll run in this course — `start-master.sh`, `start-dfs.sh`, `start-yarn.sh` — is itself just a Bash shell script. Reading and writing simple shell scripts lets you automate repetitive setup, chain commands together, and understand exactly what those built-in scripts are doing rather than treating them as magic.

```
#!/bin/bash
# check_cluster.sh -- a tiny cluster health-check script

echo "Checking Java..."
if command -v java &> /dev/null; then
    echo "  OK: $(java -version 2>&1 | head -n 1)"
else
    echo "  MISSING: install a JDK before continuing"
    exit 1
fi

echo "Checking Spark master port 7077..."
if nc -z localhost 7077; then
    echo "  OK: Spark master is listening"
else
    echo "  WARNING: nothing listening on 7077 -- is the master started?"
fi

for svc in NameNode DataNode Master Worker; do
    if jps | grep -q "$svc"; then
        echo "$svc is running"
    else
```

```

    echo "$svc is NOT running"
fi
done

```

A handful of constructs carry you a very long way: variables (name=value, referenced as \$name), conditionals (if / then / fi), loops (for ... do ... done), and command substitution (\$(command) captures a command's output into a variable). Every start-*.sh script bundled with Hadoop and Spark is built from exactly these same four building blocks, usually looped over a file listing every worker hostname.



Why this is a real job skill, not busywork

Infrastructure and data engineering roles routinely require writing small Bash utilities: health checks before a job runs, log rotation, cleanup of old temp/shuffle files, or gluing a cron schedule to a spark-submit call. Being comfortable in Bash is often assumed, unstated background knowledge in Big Data job postings — it rarely appears as its own bullet point precisely because interviewers assume anyone who's used Linux seriously already has it.

1.8 Archives & Compression — tar, gzip, and Why Big Data Cares

Every Spark and Hadoop distribution you'll download in this course arrives as a compressed archive (a .tgz / .tar.gz file), and compressed file formats (Parquet with Snappy/Gzip compression, in particular) are central to Big Data storage efficiency later in Units III-IV. Getting comfortable with tar and gzip now removes friction from day one of installation.

```

# Extract a Spark/Hadoop-style .tgz distribution
tar -xvzf spark-3.5.0-bin-hadoop3.tgz -C /opt/
#   x = extract, v = verbose, z = gzip-compressed, f = file follows

# Create your own compressed archive of a log directory before clearing it
tar -cvzf logs_backup_2026-08-12.tar.gz /opt/spark/logs/

# Peek inside an archive WITHOUT extracting it
tar -tvzf spark-3.5.0-bin-hadoop3.tgz | head

# Compress / decompress a single file directly
gzip access.log          # produces access.log.gz, removes original
gunzip access.log.gz     # reverses it

```



A preview of what's coming

In Unit IV, you'll learn that columnar file formats like Parquet apply compression PER COLUMN, which is a major reason Spark SQL queries on Parquet files are so much faster and cheaper to store than the equivalent CSV. The tar/gzip skills here are the same underlying idea (trade CPU time for smaller data on disk/network) applied at the whole-file level instead of the column level.

 Check yourself — 1.7 & 1.8

Q1. In a Bash script, what does ``$(command)`` do, and give one cluster-relevant example of when you'd use it?

Q2. Why do Spark and Hadoop distributions ship as `.tar.gz` files rather than uncompressed folders?

Q3. What's the difference between compressing a single file with `gzip` and archiving a whole directory tree with `tar`?

Hands-on Challenge:

Write a 10-15 line Bash script that loops through every `.log` file in a directory, gzips any file larger than 1MB, and prints a one-line summary of how many files it compressed. Run it against a folder of dummy log files you create with ``dd`` or simple text redirection.

MODULE 2 — Distributed Systems Fundamentals

Everything in this course — RDDs, Spark SQL's execution plans, Kafka's partitions — is a specific engineering answer to one root problem: a single computer cannot hold or process all the data a modern organization generates. This module builds the conceptual vocabulary for "why distribute at all" before Unit I hands you Spark as the tool that does it well.

2.1 What Is Distributed Computing? Vertical vs. Horizontal Scaling

Distributed computing means splitting a computation (and usually the data it operates on) across multiple independent machines — connected by a network, each with its own CPU, RAM, and disk — that coordinate to produce one combined result. The machines don't share memory directly; they share a network.

When a single machine becomes insufficient, there are exactly two directions to scale:

- Vertical scaling ("scale up") — buy a bigger machine: more CPU cores, more RAM, faster disks, in the SAME box. Simple to reason about (still one machine, no network coordination needed) but has a hard ceiling — there's a most powerful server money can buy, and after that you're stuck.
- Horizontal scaling ("scale out") — add MORE machines and spread the work across them. No theoretical ceiling — need more capacity, add another node — but it introduces real complexity: network communication, partial failures, coordinating who does what, and combining partial results correctly.

A quick numeric intuition

Suppose you need to process 10 terabytes of log data and a single strong server can process roughly 200 GB/hour. That's 50 hours end-to-end on one machine — over two days. Split identically across 50 commodity machines working in parallel and, in the ideal case with no coordination overhead, that same job finishes in about an hour. Distributed computing trades raw single-machine power for parallelism across many cheaper machines.



One Ox vs. a Team of Chickens

There's an old engineering saying: "if you're trying to plow a field, would you rather use one huge, incredibly strong ox, or 1,024 chickens?" Vertical scaling is buying a bigger and bigger ox. Eventually, no ox on Earth is strong enough, and the ox you do have costs a fortune to feed (a single top-tier server is disproportionately expensive per unit of power).

Horizontal scaling is the 1,024 chickens. Individually, each chicken is laughably weak. But if you can actually organize 1,024 chickens to pull in the same direction at the same time, you plow the field faster than any ox ever could, and each chicken is cheap and replaceable — if one chicken wanders off (a node fails), the other 1,023 keep pulling.

The entire difficulty of distributed computing is that last clause: organizing 1,024 chickens is a genuinely hard coordination problem. That coordination problem is precisely what Hadoop's MapReduce and later Spark's execution engine were built to solve for you.

Why the industry went horizontal

Beyond the physical ceiling on vertical scaling, horizontal scaling on commodity hardware is dramatically cheaper per unit of throughput, and — critically — it gives you fault tolerance almost for free. If a company's data no longer fits (or computes fast enough) on one very expensive machine, and downtime from a single point of failure is unacceptable, horizontal scaling stops being optional. This exact pressure is why Google published the original MapReduce paper in 2004, and why Hadoop, and later Spark, exist at all.

2.2 The Bottleneck Problem — Network I/O vs. Disk I/O vs. RAM Speed

Distributed systems live and die by one uncomfortable fact: not all forms of "moving data around" are equally fast. Understanding the relative speed of RAM, disk, and network is the single biggest reason Spark exists as a faster alternative to plain Hadoop MapReduce — this is literally the first bullet point of Unit I's syllabus.

Rough relative speeds (order-of-magnitude intuition)

- RAM access — roughly 100 nanoseconds for a random access. Blazing fast.
- SSD disk access — roughly 100 microseconds. About 1,000x slower than RAM.
- Spinning HDD disk access — roughly 10 milliseconds (seek time). Roughly 100,000x slower than RAM.
- Network round-trip within the same data center — roughly 0.5 milliseconds. Comparable to or slower than SSD, and this is the BEST-case network number; cross-region network calls are dramatically worse.

Classic Hadoop MapReduce writes intermediate results (the output of each Map stage) to DISK before the next stage can read them, specifically for fault tolerance — if a machine dies mid-job, the intermediate output on disk survives. Spark's headline improvement is keeping intermediate results in RAM across stages whenever possible (that's exactly what Unit I means by "in-memory processing"), only spilling to disk when memory genuinely runs out. That single design choice is why Spark benchmarks are commonly 10-100x faster than equivalent MapReduce jobs on iterative workloads.

The Office Building of Speed

Imagine you need a specific document. If it's already in your HAND (RAM), you have it instantly. If it's in the filing cabinet in your own office (SSD), it takes a short walk. If it's in the archive room in the basement (spinning HDD), it's a slow elevator ride each way. If it's in a DIFFERENT BUILDING across town that you have to call and request a courier for (network), you're at the mercy of traffic, weather, and whether that other office even answers the phone.

Classic MapReduce, after every single step of work, insists on filing EVERYTHING back down to the basement archive "just in case," then walking all the way back down to retrieve it again for the next step. Spark instead tries to just keep the document in your hand between steps whenever there's room, only reluctantly filing it away when your hands are genuinely full.

That's the entire in-memory-vs-disk story of Unit 1 in one metaphor: same building, same documents, wildly different amount of walking.

Why this drives real architecture decisions

Engineers explicitly design data pipelines around this hierarchy: keep hot, frequently-reused data in RAM (Spark's `.cache()/persist()`), keep warm data on fast local SSD, and only reach across the network for cold or one-time-use data. Network I/O in particular is treated as the enemy in distributed system design — a huge amount of Spark's internal optimization (partitioning strategy, broadcast joins, avoiding unnecessary shuffles) exists purely to minimize how much data has to cross the network between nodes, because network calls are consistently the slowest and least predictable part of the whole stack.

Check yourself — 2.1 & 2.2

- Q1. Give one genuine advantage vertical scaling still has over horizontal scaling, despite its ceiling.
- Q2. Why does writing intermediate MapReduce output to disk help with fault tolerance, and what does Spark trade away to avoid doing this as often?
- Q3. Rank RAM, SSD, spinning disk, and same-datacenter network from fastest to slowest access, and explain why network sits where it does.

Hands-on Challenge:

On your own machine, write a tiny script (any language) that (a) writes 200MB of random data to a file, reads it back, and times it, versus (b) generates the same 200MB directly into an in-memory array/list and times just reading it. Compare the two timings and connect the gap you measure to why Spark prefers RAM over disk between processing stages.

2.3 Cluster Architectures — Master/Worker Topology & Resource Managers

Nearly every distributed data system you'll meet in this course — Hadoop, Spark Standalone, YARN, Kafka — organizes its machines using some flavor of a master/worker (also called leader/follower, or driver/executor in Spark's specific vocabulary) topology.

The general pattern

- Master (coordinator) — keeps track of which workers exist, what work is available, and who's doing what. Does NOT usually do the heavy data processing itself.
- Workers (executors) — the actual muscle; they hold data partitions and execute the real computation, then report results/status back to the master.
- Resource Manager — a specialized master-like role whose only job is deciding how much CPU/RAM each application gets on the shared cluster, and where. In the Hadoop ecosystem this role is YARN (Yet Another

Resource Negotiator); Spark can run its OWN simpler resource manager in "Standalone" mode, or delegate that job to YARN, Kubernetes, or Mesos.

Spark's specific naming (a preview of Unit I)

- Driver — the process running your main program; it builds the execution plan and acts as the master for that one application.
- Cluster Manager — Standalone, YARN, or Kubernetes; hands out worker resources to the driver.
- Executors — worker processes that actually run tasks and hold cached data partitions.

```
# ----- Spark Standalone cluster: minimal manual walkthrough -----

# On the MASTER machine: start the master process (defaults to port 7077, UI on
8080)
$SPARK_HOME/sbin/start-master.sh

# Open the master's Web UI to grab its exact URL
# http://<master-ip>:8080 --> copy the 'spark://<master-ip>:7077' URL shown
there

# On EACH WORKER machine: point it at the master to register
$SPARK_HOME/sbin/start-worker.sh spark://<master-ip>:7077

# Submit an application to the cluster you just formed
spark-submit --master spark://<master-ip>:7077 \
              --executor-memory 2g --total-executor-cores 4 \
              my_job.py
```



The Restaurant Kitchen

Think of a busy restaurant kitchen. The head chef (the master / driver) doesn't personally chop every vegetable — they look at incoming orders, break each order into tasks, and shout assignments: "you, station 3, start the sauce; you, station 7, plate this."

The line cooks (workers / executors) each own their station, have their own tools and ingredients close at hand (their local memory/disk), and execute exactly the task they were assigned, then call out "done!" back to the chef.

The Resource Manager is like the restaurant's staffing coordinator, standing outside the kitchen door deciding, across MULTIPLE simultaneous kitchens sharing the same building (multiple applications sharing one physical cluster), how many line cooks each kitchen is allowed to borrow right now. In Spark Standalone mode, the head chef doubles as their own staffing coordinator; in a YARN-managed cluster, that coordinator is a separate, dedicated role.



Why resource managers matter in production

Real clusters are almost never dedicated to one single Spark job. Dozens of jobs from different teams compete for the same physical machines throughout the day. YARN (or Kubernetes, increasingly) exists

specifically to fairly and safely multiplex that shared hardware — enforcing that one team's runaway job can't starve every other team's job of memory or CPU. This is why large organizations almost never run Spark in pure Standalone mode in production; they run it ON TOP of YARN or Kubernetes precisely for this multi-tenant resource negotiation.

Check yourself — 2.3

- Q1. In Spark's own vocabulary, which piece plays the "master" role and which plays the "worker" role?
- Q2. What specific job does a Resource Manager (like YARN) do that a plain master process doesn't need to worry about on a single dedicated cluster?
- Q3. Why would a company running many different data teams on shared hardware prefer YARN/Kubernetes over Spark Standalone mode?

Hands-on Challenge:

If you have access to two machines/VMs (or can simulate with two terminal tabs on one machine using different ports), start a Spark Standalone master, register one worker against it, open the master's Web UI in a browser, and take note of how many cores/how much memory it reports as available before you've submitted any job.

2.4 Fault Tolerance, Replication, and a Gentle Intro to the CAP Theorem

Once you accept Module 2.1's premise — many cheap machines instead of one expensive one — you must also accept that individual machines WILL fail. Not "might": across a fleet of a few hundred commodity servers running continuously, disk failures, network blips, and outright crashes are a statistical certainty, not an edge case. Distributed systems are therefore designed assuming failure as the default condition, not the exception.

The two core defenses you'll see throughout this course:

- Replication — keep multiple copies of the same data on different machines (HDFS's default 3x replication from Module 3.2) so a single node's failure never means data loss.
- Recomputation from lineage — Spark's specific trick (Unit III): instead of replicating every intermediate result, Spark remembers the SEQUENCE OF OPERATIONS (the "lineage") that produced an RDD, and can simply recompute a lost partition from its lineage if a node dies, rather than needing a stored copy of every intermediate step.

A one-paragraph taste of the CAP theorem

The CAP theorem states that a distributed data store can only fully guarantee two of the following three at the same time: Consistency (every read gets the most recent write), Availability (every request gets SOME response, even during a failure), and Partition tolerance (the system keeps working even if network communication between some nodes breaks). Since network partitions are a real possibility in any distributed system, partition tolerance is effectively non-negotiable in practice, which means real systems are actually choosing between prioritizing

consistency or availability when a partition happens. You won't be asked to design around CAP in this course, but the vocabulary appears constantly in Big Data interviews and system-design discussions, so it's worth recognizing by name.



The Relay Race with Backup Runners

Picture a relay race where, instead of one runner per leg, the team sends out THREE identical runners per leg, all carrying an identical baton, running in parallel down slightly different lanes. If one runner trips and falls (a node fails), the other two are already carrying the same baton forward -- the race doesn't stop, and nobody has to run back to the start.

Spark's lineage trick is subtly different and arguably cleverer: instead of paying for three runners on every single leg, Spark just keeps a WRITTEN RECIPE of exactly how each leg's baton was made from the previous leg's baton. If a runner falls and the baton itself is lost, Spark doesn't panic -- it just re-runs that one leg's recipe again from the last known safe checkpoint, rather than having paid the ongoing cost of triple-running every leg of the whole race.



Why interviewers ask about this

"Explain how Spark achieves fault tolerance without replicating every RDD" is a genuinely common Big Data interview question, precisely because the lineage-based answer demonstrates you understand Spark's architecture rather than having memorized a feature list. Being able to contrast that with HDFS's brute-force replication approach in the same breath is exactly the kind of comparative understanding this handbook is building toward.

2.5 Data Partitioning & Parallelism

Splitting data across machines only helps if the SPLIT itself is sensible. A partition is simply a chunk of the overall dataset assigned to one task/executor to process independently. Good partitioning is the difference between a cluster where every machine works evenly (fast) and one where 9 machines finish in a second while the 10th machine chokes on a bad partition for an hour (a problem specifically called "data skew," which you'll meet again in Unit III/IV).

- Too few partitions — you're not using all your available cores; machines sit idle while a small number of huge partitions get processed.
- Too many, tiny partitions — the overhead of scheduling and coordinating thousands of tiny tasks can outweigh the actual work being done in each one.
- Uneven / skewed partitions — one partition is dramatically larger than the others (e.g., partitioning e-commerce orders by country when 80% of orders come from one country), so one task becomes the bottleneck the whole job waits on.



Slicing the Pizza for a Study Group

Imagine ordering one giant pizza for a 10-person study group and slicing it into exactly 10 equal pieces -- everyone eats at roughly the same pace and the group moves on together. That's balanced partitioning: one slice (partition) per person (core/executor), roughly equal size.

Now imagine slicing it into just 2 giant pieces for 10 people -- 8 people sit there with nothing to do while 2 people slowly work through an enormous slice each. That's too few partitions. Or imagine 200 confetti-sized slices -- everyone spends more time passing plates around and arguing over crumbs than actually eating. That's too many, over-partitioned. And if one slice accidentally gets ALL the pepperoni while the rest are plain, the pepperoni-lover takes forever finishing theirs while everyone else already left the table -- that's data skew.

2.6 Serialization — Getting Objects Across the Network

Whenever data needs to travel across the network between nodes (Module 2.2's slowest tier) or be written to disk, an in-memory object (a Java/Scala object, a Python object) has to first be converted into a flat sequence of bytes — this process is called serialization — and converted back into a usable object on the other end, called deserialization.

This matters directly in Spark: by default the JVM uses standard Java serialization, which is reliable but verbose and relatively slow. Spark offers Kryo serialization as a faster, more compact alternative, and choosing between them is a real, commonly-tuned production setting (`spark.serializer`) that directly affects shuffle performance, since every shuffle (Module 3.3) involves serializing data to send it across the network to the correct node.

```
# Enabling Kryo serialization in a Spark application (a Unit III/VI preview)
spark-submit --conf spark.serializer=org.apache.spark.serializer.KryoSerializer
my_job.py
```

✓ The mental model to keep

Serialization is simply "packing an object into a shippable box" and deserialization is "unpacking it on arrival." Every byte that crosses the network in a shuffle has been through this pack/unpack cycle, which is exactly why a faster, more compact packer (Kryo) noticeably speeds up shuffle-heavy Spark jobs.

🧠 Check yourself — 2.4, 2.5 & 2.6

- Q1. Contrast HDFS's replication-based fault tolerance with Spark's lineage-based fault tolerance in your own words.
- Q2. Why is a Consistency/Availability trade-off effectively unavoidable during a network partition, per the CAP theorem?
- Q3. Describe, using the pizza analogy or your own example, what "data skew" looks like and why it slows an entire job down.
- Q4. What does serialization have to do with network I/O being the slowest tier from Module 2.2?

Hands-on Challenge:

Pick any dataset idea (e.g., orders by country, or students by class) and sketch, on paper, both a BALANCED partitioning scheme and a SKEWED partitioning scheme across 4 partitions. For the skewed version, explain in 2-3 sentences exactly which single partition would become the bottleneck and why.

MODULE 3 — The Big Data Evolution

With Module 1's mechanics and Module 2's vocabulary in hand, this final module answers the "why did any of this need to be invented" question directly, then gives you a working refresher on HDFS and MapReduce — the exact two systems Unit I spends its opening paragraph comparing Spark against.

3.1 Why Traditional Single-Node Databases Fail at Scale

A traditional relational database (think a single PostgreSQL or MySQL server) is built around the assumption that all your data fits on the disks physically attached to one machine, and that one machine's CPU can process queries against it in acceptable time. Three failure modes appear as data grows past what one machine can reasonably hold:

- Capacity ceiling — a single server has a maximum number of disks it can physically attach. Once your dataset exceeds that (modern organizations routinely generate terabytes to petabytes daily from logs, sensors, clickstreams, transactions), there is no vertical-scaling path left, full stop.
- Throughput ceiling — even if the data technically fits, one machine has a fixed number of CPU cores and a fixed disk I/O bandwidth. A query scanning a 50 TB table on one machine is bounded by that one machine's disk read speed, no matter how well-indexed the query is.
- Single point of failure — if that one server's disk fails, dies, or the data center loses power, your data (or at minimum, your availability) is gone until it's restored from backup. There's no redundancy built into the "one machine" model by default.

Distributed storage and compute systems solve all three simultaneously: capacity scales by adding nodes, throughput scales because many disks/CPU's work in parallel, and fault tolerance comes from deliberately storing multiple copies of data across different machines (and often different physical racks), so one failure never means data loss.



The One-Librarian Library

Picture a library with exactly one librarian, one front desk, and one building. That's your traditional single-node database. It works beautifully — until the town's population triples. Now there are too many books for one building to hold (capacity ceiling), too many patrons for one librarian to check out books fast enough (throughput ceiling), and if that one building floods, every book in town is gone (single point of failure).

The Big Data solution isn't to build one impossibly huge library with an impossibly fast librarian. It's to open a NETWORK of smaller branch libraries across town, each holding a portion of the collection, each with its own librarian, and — crucially — keeping a duplicate copy of every book on at least two different branches, so a flood at one branch never actually loses a book.



Real-world scale that forced this shift

Companies like Google, Yahoo, and Facebook in the mid-2000s were generating web crawl data, search logs, and social graphs measured in petabytes — orders of magnitude beyond what any single database server of that era could store, let alone query. That exact pressure is what produced Google's 2003 Google File System paper and 2004 MapReduce paper, which the open-source community then reimplemented as Hadoop (HDFS + Hadoop MapReduce) — the direct ancestor of everything in this course.

3.2 HDFS Refresher — NameNode & DataNode

HDFS (Hadoop Distributed File System) is the storage layer under classic Hadoop, and it's still commonly used as Spark's storage layer too. It follows exactly the master/worker pattern from Module 2:

- NameNode (master) — holds the FILESYSTEM METADATA ONLY: the directory tree, file names, and — critically — which DataNodes hold which blocks of each file. It does NOT store the actual file contents.
- DataNode (worker) — stores the actual data, physically broken into fixed-size blocks (128 MB by default in modern Hadoop). Each block is REPLICATED across multiple DataNodes (default replication factor: 3) specifically for fault tolerance.
- Block — the unit HDFS splits every file into. A 500 MB file with a 128 MB block size becomes 4 blocks (128+128+128+116), each stored on (by default) 3 different DataNodes.

When a client wants to read a file, it first asks the NameNode "which DataNodes hold the blocks of this file?", then talks DIRECTLY to those DataNodes to stream the actual bytes — the NameNode is never in the data path itself, only the metadata path. This is precisely why the NameNode can be a single (carefully protected) coordinator even for a cluster storing petabytes: it's only ever handling small metadata requests, not the heavy data transfer.



The Library Card Catalog

The NameNode is the library's central card catalog. It doesn't hold a single book itself — it just knows that 'Chapter 1 of Moby Dick is on Shelf B at Branch 3, and also on Shelf F at Branch 9, and also on Shelf A at Branch 14' (three replicas, three different branches).

When you (the reader/client) want the book, you check the catalog once to learn WHICH branches have it, then you walk directly to the nearest branch yourself to actually read it — the catalog desk never hands you the book personally, and never gets clogged up handling the actual reading traffic.

If Branch 9 burns down overnight, the catalog notices Branch 9 stopped checking in, quietly points future readers to Branches 3 and 14 instead, and — behind the scenes — asks some other branch to make a fresh replacement copy so the collection stays at 3 replicas system-wide.

3.3 MapReduce Execution Flow Refresher

MapReduce is the classic Hadoop programming model for processing data stored in HDFS, structured as exactly two user-defined functions plus a system-managed step in between:

1. Map — runs once per input record (or block), transforming raw input into intermediate key-value pairs. Map tasks run in parallel, one per HDFS block, ideally on the SAME machine that already physically holds that block (this is called "data locality" — moving the tiny bit of code to the data, instead of moving the huge data to the code).
2. Shuffle & Sort (automatic, not user-written) — the framework groups all intermediate values that share the same key together, across ALL map tasks, and sorts them, then routes each key's group to exactly one Reduce task. This step is network- and disk-heavy — it's the single most expensive phase of a MapReduce job.
3. Reduce — runs once per unique key (or key group), receiving ALL values for that key and combining them into a final output, which is written back to HDFS.

Concrete example: classic word count

```

INPUT (split across HDFS blocks, processed by parallel Map tasks):
  Block A: "spark is fast spark is fun"
  Block B: "hadoop is reliable"

MAP output (each Map task emits key-value pairs independently, in parallel):
  Task A -> (spark,1) (is,1) (fast,1) (spark,1) (is,1) (fun,1)
  Task B -> (hadoop,1) (is,1) (reliable,1)

SHUFFLE & SORT (framework groups identical keys together across ALL tasks):
  spark    -> [1, 1]
  is       -> [1, 1, 1]
  fast     -> [1]
  fun      -> [1]
  hadoop   -> [1]
  reliable -> [1]

REDUCE output (one Reduce task per key, sums the list):
  spark -> 2
  is    -> 3
  fast  -> 1
  fun   -> 1
  hadoop-> 1
  reliable -> 1

```

This is exactly Unit I's opening complaint

Notice that between EVERY stage above (Map -> Shuffle -> Reduce, and again if you chain multiple MapReduce jobs for a more complex pipeline), classic Hadoop MapReduce writes intermediate results to DISK, then reads them back — recall Module 2.2's speed hierarchy. Spark's core innovation, which Unit I introduces next, is running this same kind of Map/Reduce-style logic while keeping intermediate results in memory (as RDDs) across stages whenever possible, which is exactly why Spark is commonly 10-100x faster on iterative or multi-stage jobs.

Where MapReduce still matters

Even though Spark has largely replaced hand-written MapReduce for new development, the CONCEPTS — data locality, key-based partitioning, a shuffle phase, fault tolerance through replication — are directly inherited by Spark's own execution model. When you learn Spark's shuffle behavior in later units (and specifically why operations like groupByKey are expensive), you are learning a direct descendant of exactly this Map -> Shuffle -> Reduce pattern.

Check yourself — 3.1, 3.2 & 3.3

Q1. Why does the HDFS NameNode not become a bottleneck even for petabyte-scale clusters, given that it's a single coordinator?

Q2. In the word-count example, which phase (Map, Shuffle, or Reduce) would you expect to generate the most network traffic, and why?

Q3. Connect Module 2.2's RAM/disk/network speed hierarchy directly to why Spark improves on classic MapReduce's design.

Hands-on Challenge:

By hand (pen and paper or a text file), trace the full Map -> Shuffle -> Reduce flow for the sentence "spark spark is is is fun" split into two blocks of your own choosing, exactly like the worked example above, ending with the final reduced key-count pairs. Then write one paragraph explaining, in your own words, at which exact step Spark would instead keep the intermediate data in RAM.

3.4 A Short Timeline — How We Got From Hadoop to Spark to Kafka

Seeing the rough chronology helps the syllabus's unit ordering make sense — each technology was a direct reaction to the previous one's limitations:

- 2003-2004 — Google publishes the Google File System and MapReduce papers, describing how Google processes web-scale data internally.
- 2006 — Hadoop is created (originally part of the Nutch search project), open-sourcing an implementation of GFS (as HDFS) and MapReduce for the wider world.
- 2008-2012 — Hadoop MapReduce becomes the dominant Big Data batch-processing standard; the ecosystem grows Hive, Pig, HBase, and others around it (Module 3.5).
- 2009-2014 — Spark is created at UC Berkeley's AMPLab specifically to address MapReduce's disk-heavy, batch-only limitations (Module 2.2 and 3.3's core complaint) using in-memory RDDs; it becomes an Apache top-level project in 2014.
- 2011 — Kafka is created at LinkedIn to handle high-throughput, real-time event streams (user activity, metrics) that batch-oriented MapReduce simply wasn't built for; it later becomes an Apache project too.

- 2014-onward — Spark SQL, Structured Streaming, and MLlib mature, YARN and later Kubernetes become the dominant resource managers, and Spark + Kafka together become a standard combination for real-time Big Data pipelines — exactly the combination Units IV-VI of this course build toward.



Three Generations of the Same Family Business

Think of this as a three-generation family business. Grandpa Hadoop built the original warehouse (HDFS) and the original assembly line (MapReduce) -- reliable, but the assembly line insists on double-checking and re-filing paperwork (writing to disk) after every single station, which makes it slow for anything that needs to loop back and reprocess the same goods.

Dad Spark inherited the warehouse (still happily reads from HDFS) but redesigned the assembly line to keep goods on the conveyor belt (in memory) between stations instead of re-filing everything, and made the whole line dramatically faster for iterative and interactive work.

Cousin Kafka runs an entirely different but related business next door: instead of processing a warehouse of ALREADY-STORED goods, Kafka is the courier company built for handling a continuous, never-ending stream of packages arriving in real time -- and by Unit V, you'll see Dad Spark and Cousin Kafka's businesses wired directly together on the same loading dock.

3.5 A Quick Tour of the Hadoop Ecosystem

Unit I explicitly asks you to "compare Spark vs. Hadoop ecosystem," so it helps to know the ecosystem isn't just "Hadoop" as one monolithic thing — it's a constellation of tools that grew up around HDFS and MapReduce, each solving one specific pain point:

- HDFS — the distributed storage layer (Module 3.2); the foundation almost everything else sits on.
- MapReduce (Hadoop's own) — the original distributed batch processing engine (Module 3.3), which Spark is most directly compared against.
- YARN — the resource manager (Module 2.3) that lets multiple frameworks (MapReduce, Spark, and others) share one physical cluster.
- Hive — lets you write SQL-like queries (HiveQL) that get translated into MapReduce (or, today, often Spark) jobs under the hood, so analysts don't have to hand-write MapReduce code.
- Pig — a scripting language (Pig Latin) for expressing data transformation pipelines, an alternative to hand-written MapReduce, largely superseded by Spark today.
- HBase — a distributed, real-time NoSQL database built on top of HDFS, for fast random read/write access that plain HDFS (built for large sequential scans) isn't designed for.
- Sqoop — a tool for bulk-importing data from traditional relational databases (MySQL, Oracle) into HDFS, and exporting back out.
- Oozie — a workflow scheduler for chaining multiple Hadoop/Spark jobs together into a pipeline with dependencies ("run job B only after job A succeeds").

Spark is best understood not as a replacement for the entire ecosystem, but primarily as a much faster, more general-purpose REPLACEMENT FOR THE MAPREDUCE ENGINE specifically — it still commonly reads from HDFS, still commonly runs under YARN, and increasingly replaces Hive's and Pig's execution engines underneath while keeping similar SQL-like interfaces (Spark SQL, in Unit IV).

Why this comparison is a favorite interview topic

"Is Spark a replacement for Hadoop?" is a deliberately imprecise-sounding interview question that's actually testing whether you understand the ecosystem's layers. The precise answer — Spark replaces the MapReduce PROCESSING engine, not HDFS storage or YARN resource management, and often runs happily alongside both — is exactly the kind of nuanced answer that distinguishes a candidate who understands the architecture from one who's only memorized "Spark is faster than Hadoop."

3.6 Batch vs. Real-Time Analytics — A Preview of Unit I's Opening Line

Unit I's syllabus explicitly opens with "comparison of batch vs. real-time analytics," so it's worth pre-loading the core distinction now:

- Batch processing — data is collected over a period of time (an hour, a day), then processed all at once as a finite, bounded dataset. Classic MapReduce jobs and most traditional data warehouse ETL pipelines are batch. Latency from event to insight is typically minutes to hours, sometimes longer.
- Real-time (stream) processing — data is processed continuously, record by record or in small "micro-batches," as it arrives, with latency measured in milliseconds to seconds. Spark Streaming / Structured Streaming (Unit V, paired with Kafka) and Kafka Streams are real-time processing tools.

Neither approach is strictly "better" — a monthly financial reconciliation report is naturally a batch job (correctness and completeness matter more than speed), while fraud detection on a credit card swipe needs a real-time answer within milliseconds or the transaction has already gone through. Recognizing which category a given business requirement falls into is a foundational Big Data architecture skill you'll keep exercising through Unit V.

Check yourself — 3.4, 3.5 & 3.6

Q1. In one or two sentences each, explain why Spark was created and why Kafka was created — what specific gap did each fill?

Q2. Precisely which layer of the Hadoop ecosystem does Spark most directly replace, and which layers does it typically continue to rely on?

Q3. Give one real-world example (other than the ones in this handbook) of a task that's naturally batch, and one that's naturally real-time, and justify each choice.

Hands-on Challenge:

Write a half-page comparison table (on paper or in a text file) with three columns -- Hadoop MapReduce, Spark, Kafka -- and one row each for: primary purpose, typical latency, and where the data

lives while being processed. Filling this in from memory, without looking back at this handbook, is a strong sign you're ready for Unit I.

MODULE 4 — Career Opportunities & Industry Exposure

Cluster Computing is not just an academic unit — the skills stacked up in this handbook and across INT315 sit directly underneath some of the highest-demand, best-compensated roles in modern software and data careers. This module maps what you're learning onto real job titles, real company use-cases, and a realistic path forward from "I finished this course" to "I'm employable in this space."

4.1 Job Roles This Course Feeds Into

- Big Data Engineer / Data Engineer — designs and builds the pipelines that move and transform data at scale, typically using Spark (batch and streaming), Kafka, and cloud storage (S3/HDFS/GCS). This is the most direct match for INT315's syllabus.
- Data Platform / Cluster Engineer — owns the health of the cluster itself: provisioning nodes, tuning YARN/Kubernetes resource allocation, capacity planning, and keeping Spark/Hadoop/Kafka services running reliably. Builds heavily on Module 1 and 2 of this handbook.
- Machine Learning Engineer (Big Data track) — trains and deploys models on datasets too large for a single machine, using Spark MLlib/PySpark (Unit VI) as the entry point into distributed ML.
- Data Scientist (at scale) — analysts and scientists at large companies increasingly need to query and model data directly in Spark/Spark SQL rather than pulling everything into a laptop-sized pandas DataFrame.
- Site Reliability Engineer (SRE) / DevOps for Data Platforms — applies Module 1's Linux, networking, and process-management skills specifically to keeping distributed data infrastructure available and performant.
- Solutions Architect (Big Data / Cloud) — designs end-to-end Big Data architectures for clients or internal teams, requiring the breadth this whole handbook builds: storage, compute, streaming, and cost trade-offs together.

4.2 Where This Actually Gets Used — Industry Scenarios

- Streaming recommendation engines — platforms like video/music streaming services process real-time watch/listen events through Kafka into Spark Streaming to update "recommended for you" rows within seconds of a user's last action.
- Fraud & anomaly detection — banks and payment processors score transactions in near real-time using Spark Streaming + MLlib models, needing exactly the low-latency stream processing from Module 3.6.
- Ride-hailing & logistics — matching riders to drivers, or optimizing delivery routes, at city-then-global scale requires exactly the horizontal-scaling and partitioning concepts from Module 2.1 and 2.5, processed continuously via streaming pipelines.
- Retail & e-commerce analytics — nightly batch jobs (Module 3.6) aggregate the day's sales, inventory, and clickstream data across every store/warehouse into dashboards executives read the next morning.

- Telecom network monitoring — cell towers and network equipment emit constant metric streams; Kafka + Spark Streaming pipelines detect outages or degraded performance in near real-time across a national network.
- Healthcare & genomics — Spark's parallel processing is used to run computationally heavy genomic analysis pipelines across clusters, cutting processing time from days to hours.



From Classroom Chickens to Real Flocks

Remember Module 2.1's 1,024 chickens plowing a field? In industry, those chickens have real names: a fleet of EC2/GCP instances processing a streaming service's watch-events, or a bank's fraud-detection cluster scoring a million transactions an hour. The metaphor doesn't stop being true once you graduate - it just gets a much bigger, much more expensive field to plow, with real revenue and real customer trust riding on the chickens staying coordinated.

4.3 Certifications & Learning Paths Worth Knowing About

You don't need any of these to pass INT315, but they're commonly recognized signals in the Big Data job market and give you a concrete "what's next" after this course:

- Databricks Certified Associate Developer for Apache Spark — arguably the most directly relevant certification to this exact course; tests hands-on Spark DataFrame/SQL knowledge.
- Cloudera Data Platform certifications — historically well-known in the Hadoop ecosystem space, still referenced in many job postings.
- Cloud provider Big Data / Data Engineer certifications — AWS Certified Data Engineer, Google Cloud Professional Data Engineer, Microsoft Azure Data Engineer Associate — these validate running Spark-based pipelines on managed cloud services (EMR, Dataproc, Databricks-on-Azure).
- Apache Kafka-focused credentials (e.g., Confluent's certifications) — useful once you're specifically leaning into the real-time streaming side from Unit V.



How recruiters actually read this course on a resume

A line like "Cluster Computing — Spark, Scala, RDDs, Spark SQL, Spark Streaming with Kafka, Spark MLlib" signals to a technical recruiter or hiring manager that a candidate has touched the full modern Big Data stack, not just theory. Pairing that coursework with even one small personal project (e.g., a Spark job processing a public dataset, pushed to GitHub with a clear README) is consistently what turns a transcript line into an actual interview callback.

4.4 A Realistic Skill Roadmap After INT315

4. Finish this course solidly, actually running every hands-on exercise rather than only reading them — employers and interviewers can tell the difference.

5. Build one small end-to-end personal project: ingest a public dataset, process it with Spark (batch), and if time allows, simulate a streaming version with Kafka.
6. Get comfortable with at least one cloud platform's managed Big Data service (AWS EMR, GCP Dataproc, or Databricks Community Edition, which is free) so you're not limited to a local single-node setup.
7. Learn just enough SQL and Python/Scala fluency to be dangerous — Spark SQL (Unit IV) and PySpark (Unit VI) are the two most commonly requested skills in entry-level Big Data job postings.
8. Practice explaining your own project out loud in the same five-beat structure this handbook uses: what problem, what architecture, what trade-off, what result — that narrative is exactly what interviewers are listening for.

MODULE 5 — Interview Preparation: 50 Q&A on Prerequisites & Foundations

The questions below are written the way an experienced Big Data / Cluster Computing interviewer would actually ask them — plain, sometimes deceptively simple-sounding, testing whether you truly understand the concept rather than whether you've memorized a definition. Each answer models the kind of confident, precise response that leaves a strong impression. These questions are drawn strictly from the Prerequisites & Foundations material in Modules 1-3 above.

A. Linux, Shell & Environment (Q1–Q12)

Q1. What's the difference between an absolute path and a relative path in Linux, and why does it matter for scripts like `start-master.sh`?

Best answer: An absolute path always starts from root (`/`) and points to exactly one location regardless of where you currently are, e.g. `/opt/spark/bin/spark-shell`. A relative path is resolved against your current working directory, e.g. `bin/spark-shell` only works if you're already standing inside `/opt/spark`. Cluster startup scripts deliberately use absolute paths (often built from an environment variable like `SPARK_HOME`) internally precisely so they work correctly no matter which directory the person running them happens to be in.

Q2. Explain `PATH` in your own words, and describe exactly what happens when you type a command that isn't found.

Best answer: `PATH` is an ordered list of directories the shell searches, in order, whenever you type a bare command name. When you run `spark-shell`, the shell walks each directory in `PATH` looking for an executable file with that exact name, and runs the first match it finds. If it reaches the end of the list without a match, it returns 'command not found' -- which doesn't mean the file doesn't exist anywhere on disk, only that none of the directories currently listed in `PATH` contain it.

Q3. Why does exporting a variable in one terminal session not persist after you close that terminal?

Best answer: `export` makes a variable part of the current shell process's environment, which is only inherited by that shell and any child processes it launches. When the terminal closes, that entire process (and its environment) is destroyed. To make a variable available in every future session, it has to be written into a shell startup file like `~/.bashrc` or `~/.zshrc`, which the shell re-reads and re-executes every time a new interactive session starts.

Q4. What's the practical difference between `JAVA_HOME` and simply having java available on `PATH`?

Best answer: Having java on `PATH` only guarantees that typing 'java' launches the JVM. Many tools, including Spark, additionally need to know the `ROOT` of the JDK installation to locate other resources bundled alongside it -- headers, additional binaries, libraries. `JAVA_HOME` provides that root explicitly rather than making every tool independently reverse-engineer it from wherever 'java' happens to resolve to on `PATH`, which can be an unreliable symlink chain.

Q5. A colleague says 'just `chmod 777` the folder and it'll work.' Why is this bad practice on a cluster machine?

Best answer: `chmod 777` grants read, write, and execute permission to the owner, the group, AND every other user on the machine -- effectively removing all access control. On a shared or production cluster machine, this means any other user or compromised process could read, modify, or delete files it has no business touching. The correct fix is almost

always to identify and correct the actual OWNER or GROUP mismatch (often via `chown`) rather than opening permissions to everyone.

Q6. What does the `-f` flag do in `tail -f app.log`, and when would you use it operationally?

Best answer: `-f` means 'follow' -- instead of printing the last lines and exiting, `tail` keeps the file open and streams new lines to the terminal as they're written. It's the standard way to watch a Spark or Hadoop daemon's log file live while a job runs, letting you see errors or stage progress the moment they're written rather than repeatedly re-running `tail`.

Q7. Why might a production team prefer SSH key-based login over password login for cluster automation?

Best answer: Key-based login lets a master node or automation script authenticate to every worker node without a human typing a password interactively, which is essential since cluster startup scripts need to log into potentially dozens of machines unattended. It's also more secure: private keys are far harder to brute-force than passwords, and access can be revoked per-key without changing a shared password across the whole fleet.

Q8. What's the risk of binding a cluster service to `0.0.0.0` versus `127.0.0.1`, and when is each appropriate?

Best answer: Binding to `0.0.0.0` makes the service reachable from any network interface, which is necessary for genuine multi-machine clusters where workers on other machines must connect in -- but it also means the service is reachable by anything else that can route to that machine, which is why firewalls matter. Binding to `127.0.0.1` restricts connections to the same machine only, which is safer for local single-node testing but will silently break a multi-machine cluster, since remote workers simply cannot reach a loopback-bound service.

Q9. Explain, step by step, what happens in memory when the JVM's Garbage Collector runs.

Best answer: The GC scans the heap to determine which objects are still reachable from active references (variables, static fields, thread stacks) and which are not. Unreachable objects are reclaimed, freeing their memory for future allocations. Depending on the GC algorithm, this can require briefly pausing some or all application threads (a 'stop-the-world' pause) so the object graph doesn't change mid-scan, which is exactly why long or frequent GC pauses show up as unpredictable latency in a running application like a Spark executor.

Q10. Why would a system deliberately use off-heap memory even though it adds manual management overhead?

Best answer: Off-heap memory is invisible to the garbage collector, so allocating large buffers there avoids adding GC scanning and pause overhead for that memory -- which matters a lot when you're holding gigabytes of cached data, as Spark executors often are. The trade-off is that off-heap memory isn't automatically reclaimed; the application itself must track and free it, which is exactly why it's an opt-in, carefully configured setting (`spark.memory.offHeap`) rather than the default.

Q11. What's the difference between `kill <pid>` and `kill -9 <pid>`, in terms of what the target process actually experiences?

Best answer: `kill <pid>` sends `SIGTERM`, a polite request asking the process to shut down; a well-behaved process can catch this signal and clean up open files, flush buffers, or finish an in-flight write before exiting. `kill -9` sends `SIGKILL`, which the operating system enforces immediately and unconditionally -- the process has no opportunity to run any cleanup code at all, which can leave data half-written or resources locked, so it should only be used when a normal kill has failed to stop an unresponsive process.

Q12. Why do Spark and Hadoop distributions ship as `.tar.gz` archives instead of plain uncompressed folders?

Best answer: A .tar.gz bundles many files and directories into a single file (tar) and then compresses that single file (gzip), which dramatically reduces download size and transfer time across the internet compared to shipping an uncompressed folder tree with the same content. It's simply a more efficient way to distribute a large, deeply-nested set of files as one downloadable artifact.

B. Distributed Systems Fundamentals (Q13–Q28)

Q13. In your own words, what is distributed computing, and how is it different from just running a program on a powerful server?

Best answer: Distributed computing splits a computation, and typically the data it operates on, across multiple independent machines connected over a network, each contributing its own CPU, RAM, and disk, that coordinate to produce one combined result. Running on a single powerful server, no matter how strong, keeps everything within one machine's memory and CPU boundary and shares no coordination problem across a network -- which is exactly the complexity distributed computing deliberately takes on in exchange for a scaling ceiling that doesn't exist.

Q14. What is the fundamental limitation of vertical scaling that horizontal scaling avoids?

Best answer: Vertical scaling is bounded by the most powerful single machine that currently exists and that you can afford -- there is always a ceiling, both physical (maximum CPU sockets, RAM slots, disk bays on one motherboard) and financial (cost rises non-linearly as you approach the top of the market). Horizontal scaling has no such ceiling in principle: if more capacity is needed, you simply add another commodity machine to the pool.

Q15. Give one genuine advantage that vertical scaling still holds over horizontal scaling.

Best answer: Vertical scaling avoids network coordination entirely -- everything happens within one machine's shared memory, so there's no shuffle, no partial-failure handling, and no distributed consensus problem to solve, which makes both the system's design and its performance characteristics far simpler and more predictable for workloads that genuinely fit on one machine.

Q16. Rank RAM, SSD, spinning disk, and same-datacenter network access from fastest to slowest, and explain the practical consequence for a distributed system.

Best answer: From fastest to slowest: RAM, then SSD, then network (same-datacenter round trip), then spinning HDD -- though network and SSD are close enough that ordering can flip slightly by workload. The practical consequence is that distributed systems are designed to minimize data movement across the slowest tiers whenever possible: keep hot data in RAM, avoid unnecessary network shuffles, and only fall back to disk or cross-node network calls when genuinely required.

Q17. Why does classic Hadoop MapReduce write intermediate results to disk between stages, and what does this cost it?

Best answer: Writing intermediate output to disk after each stage means that if a machine crashes mid-job, the already-completed stage's output survives and doesn't need to be recomputed from scratch -- this is a deliberate fault-tolerance choice. The cost is speed: every stage boundary pays the price of a disk write followed by a disk read, which is dramatically slower than simply keeping that same data in RAM, especially for jobs that iterate over the same data multiple times.

Q18. How does Spark avoid paying the same disk-write cost between every stage, and what does it use instead to stay fault tolerant?

Best answer: Spark keeps intermediate results in memory as RDDs across stages whenever there's room, only spilling to disk under genuine memory pressure. For fault tolerance, instead of relying on replicated disk copies of every intermediate step, Spark tracks the LINEAGE -- the sequence of transformations that produced each RDD -- so a lost partition can simply be recomputed from that lineage rather than needing to have been durably stored in the first place.

Q19. Explain the master/worker pattern in your own words, using any two of Hadoop, Spark Standalone, or Kafka as your example.

Best answer: In the master/worker pattern, one coordinating process tracks what work exists and which workers are available, while the actual data processing happens on separate worker processes. In Spark Standalone, the Master tracks registered Workers and hands out resources to the Driver; the Workers run the actual Executors that do the computation. In HDFS, the NameNode plays an analogous coordinating role over DataNodes, though it coordinates storage metadata rather than active computation.

Q20. What specific job does a resource manager like YARN do that a basic master process doesn't?

Best answer: A basic master (like Spark Standalone's) typically coordinates resources for just one framework's own workers. A resource manager like YARN sits a layer above that, allocating CPU and memory across MULTIPLE DIFFERENT frameworks or applications (Spark jobs, MapReduce jobs, and others) that are all sharing the same physical cluster simultaneously, enforcing fairness so no single application can starve the others of resources.

Q21. Why do most real production clusters run Spark on top of YARN or Kubernetes rather than in pure Standalone mode?

Best answer: Standalone mode's built-in master is a simple resource manager designed for a cluster dedicated to Spark alone. Production clusters are almost always multi-tenant -- shared across several teams and job types -- and need a resource manager capable of fairly negotiating shared hardware across many different applications at once, which is exactly the role YARN or Kubernetes plays and Standalone mode was never designed for.

Q22. What is 'data locality' in the context of MapReduce, and why does it matter for performance?

Best answer: Data locality means scheduling a Map task to run on the SAME machine (or at least the same rack) that already physically stores the HDFS block it needs to process, rather than pulling that block across the network to wherever the task happens to be scheduled. It matters because it avoids network I/O -- one of the slowest tiers in the system -- for the initial read of the input data, which at scale meaningfully reduces both time and network congestion.

Q23. Describe, step by step, what happens during the Shuffle phase of MapReduce, and why it's typically the most expensive part of a job.

Best answer: After the Map phase produces key-value pairs independently on many machines, the Shuffle phase must gather every value that shares the same key together onto one machine (so a single Reduce task can process that key's full list), which usually requires transferring data across the network between nodes, followed by sorting it. It's expensive because it combines the two costliest operations in the system -- network transfer and disk-based sort/spill of potentially large volumes of intermediate data -- in a single phase that every MapReduce job must pass through.

Q24. What is a single point of failure, and how does distributed storage like HDFS specifically defend against it?

Best answer: A single point of failure is any one component whose failure alone brings down the whole system or causes data loss. HDFS defends against this at the data layer by replicating every block across multiple DataNodes (three copies by default) on different machines, so the failure of any one DataNode never results in losing access to the data -- the NameNode simply routes future reads to one of the remaining healthy replicas.

Q25. Why is the HDFS NameNode not a performance bottleneck even though it's a single coordinating process for a petabyte-scale cluster?

Best answer: The NameNode only ever handles small METADATA requests -- which DataNodes hold which blocks -- never the actual bulk data transfer itself. Once a client learns which DataNodes to contact, it communicates directly

with those DataNodes for the actual read or write, so the NameNode's workload stays proportional to the number of metadata operations, not to the volume of data flowing through the cluster.

Q26. What would happen, historically, if the HDFS NameNode itself failed, and why was that considered a real architectural weakness?

Best answer: In early Hadoop versions, the NameNode was a genuine single point of failure for METADATA specifically: if it went down, the cluster's data was still physically intact on the DataNodes, but clients had no way to look up which blocks belonged to which files, effectively making the whole cluster unusable until the NameNode was restored. This weakness is exactly why later Hadoop versions introduced NameNode High Availability, running a standby NameNode ready to take over.

Q27. Explain the CAP theorem in one or two sentences, and why 'partition tolerance' is effectively non-negotiable in real systems.

Best answer: The CAP theorem states a distributed system can only fully guarantee two of Consistency, Availability, and Partition tolerance at once. Since network partitions -- some nodes temporarily unable to communicate with others -- are a realistic possibility in any real network, giving up partition tolerance isn't a practical option, so real systems are actually choosing, during a partition, whether to prioritize consistency or availability.

Q28. What is data skew, and why can it make a distributed job slower than expected even with plenty of total cluster capacity?

Best answer: Data skew is when the workload is unevenly distributed across partitions -- one partition holds dramatically more data or work than the others. Because a distributed job typically can't finish until its SLOWEST task finishes, a single oversized, skewed partition becomes the bottleneck the entire job waits on, even if every other machine in the cluster finished its share of the work almost immediately and now sits idle.

C. Big Data Evolution, HDFS, MapReduce & Ecosystem (Q29–Q50)

Q29. Why do traditional single-node relational databases eventually fail as data volume grows, even on very powerful hardware?

Best answer: They hit three ceilings at once: a physical capacity ceiling (a finite number of disks one machine can attach), a throughput ceiling (a finite number of CPU cores and a finite disk I/O bandwidth bounding how fast any single machine can scan or query the data), and they represent a single point of failure with no built-in redundancy. Distributed systems solve all three simultaneously by adding nodes for capacity and throughput and replicating data for fault tolerance.

Q30. What specific real-world pressure led Google to publish the GFS and MapReduce papers that Hadoop was later built from?

Best answer: Google was processing web crawl data and search index data at a scale -- many petabytes -- that was orders of magnitude beyond what any single database server of that era could store or process, so they built their own distributed file system (GFS) and distributed processing model (MapReduce) internally, then published the design in 2003 and 2004, which the open-source community subsequently reimplemented as HDFS and Hadoop MapReduce.

Q31. Describe the role of the NameNode and the role of the DataNode, and clarify exactly which one holds the actual file bytes.

Best answer: The DataNode holds the actual file contents, physically split into fixed-size blocks and replicated across multiple DataNodes. The NameNode holds only the filesystem's metadata -- the directory structure, file names, and a mapping of which DataNodes store which blocks of each file -- and is never in the path of the actual data transfer itself.

Q32. Why does HDFS split files into fixed-size blocks (128MB by default) rather than storing each file as one contiguous unit on one machine?

Best answer: Splitting into blocks lets a single large file be spread across many DataNodes, enabling parallel reads and writes (many machines serving different pieces of the same file simultaneously) and allowing a file far larger than any single disk to be stored across the cluster as a whole. It also lets replication happen at the block level, so different blocks of the same file can even be replicated across different, load-balanced sets of DataNodes.

Q33. Walk through what happens when a client reads a file from HDFS, from the very first request to receiving the actual bytes.

Best answer: The client first contacts the NameNode asking for the block locations of the requested file. The NameNode responds with a list of blocks and, for each block, which DataNodes hold a replica of it. The client then connects DIRECTLY to the nearest or best-performing DataNode for each block and streams the actual bytes from there -- the NameNode is only involved in that initial metadata lookup, never in the actual data transfer.

Q34. What is the default HDFS replication factor, and what specific failure scenario is it designed to survive?

Best answer: The default replication factor is 3 -- every block is stored on three different DataNodes, typically spread across different racks. This is designed to survive the loss of any single DataNode (disk failure, machine crash, or even an entire rack losing power) without any data becoming unavailable, since at least one other replica remains reachable.

Q35. Define the Map function's role in MapReduce in one sentence, using your own words.

Best answer: The Map function runs independently, in parallel, once per input record or block, transforming raw input data into a set of intermediate key-value pairs without needing any knowledge of what any other Map task is doing at the same time.

Q36. Define the Reduce function's role in MapReduce in one sentence, and explain what guarantee it relies on from the Shuffle phase.

Best answer: The Reduce function runs once per unique key, receiving the COMPLETE list of all values associated with that key, and combines them into a final output; it relies on the Shuffle phase's guarantee that every value sharing a key -- no matter which Map task originally produced it -- has been correctly gathered together before Reduce ever begins processing that key.

Q37. In the classic word-count example, why can every Map task run completely independently and in parallel with no coordination?

Best answer: Because each Map task only reads its own local input split and emits key-value pairs based purely on that local data -- it never needs to know what any other Map task is producing, so there's no shared state or coordination required between them until the Shuffle phase deliberately gathers their combined output together by key.

Q38. Why is MapReduce described as primarily a BATCH processing model, and what would make it a poor fit for real-time requirements?

Best answer: MapReduce operates over a finite, already-collected dataset -- it reads a complete input, processes it fully through Map/Shuffle/Reduce, and produces a complete output, with no built-in concept of continuously arriving data. Its per-stage disk-write overhead and job startup latency (typically seconds to minutes before any output appears) make it fundamentally unsuited for use cases needing millisecond-to-second latency, like fraud detection on a live transaction.

Q39. Precisely which part of the Hadoop ecosystem does Spark most directly aim to improve upon or replace?

Best answer: Spark most directly targets the MapReduce PROCESSING ENGINE specifically -- replacing its disk-heavy, batch-only execution model with in-memory, more general-purpose processing. Spark commonly continues to rely on HDFS for storage and YARN (or Kubernetes) for resource management rather than replacing those layers.

Q40. What does Hive add on top of raw MapReduce, and who is it primarily designed for?

Best answer: Hive lets users write SQL-like queries (HiveQL) which Hive then translates into MapReduce jobs (or, in modern deployments, often Spark jobs) automatically under the hood. It's primarily designed for data analysts who are comfortable with SQL but don't want to hand-write low-level MapReduce or Spark code themselves.

Q41. Why was HBase created when HDFS already exists as a distributed storage system?

Best answer: HDFS is optimized for large, sequential reads and writes of big files -- it's a poor fit for fast, random, single-record read/write access patterns. HBase was built on top of HDFS specifically to provide fast random-access reads and writes at the individual-record level, functioning as a distributed NoSQL database rather than a general-purpose file system.

Q42. What problem does Sqoop solve in a typical enterprise Big Data pipeline?

Best answer: Most enterprises already have substantial data sitting in traditional relational databases (MySQL, Oracle, and similar). Sqoop provides a standardized, efficient way to bulk-import that existing relational data into HDFS for large-scale processing, and to export processed results back out to a relational database, bridging the traditional and Big Data worlds.

Q43. What role does Oozie play, and why is it necessary once a data pipeline involves more than one job?

Best answer: Oozie is a workflow scheduler that lets you define dependencies between multiple Hadoop/Spark jobs - for example, 'only run job B after job A completes successfully.' It's necessary because real pipelines are rarely a single

isolated job; they're typically chains of dependent steps (ingest, clean, transform, aggregate) that need to run in a specific, reliable order, often on a schedule.

Q44. Contrast batch processing and real-time (stream) processing in terms of latency and give one example use case for each.

Best answer: Batch processing collects data over a period and processes it as one finite, complete dataset, with latency typically ranging from minutes to hours -- a nightly retail sales aggregation report is a natural fit. Real-time stream processing handles data continuously as it arrives, with latency in milliseconds to seconds -- fraud detection on a live credit card transaction is a natural fit, since a batch delay of even a few minutes would let a fraudulent transaction complete.

Q45. Why was Kafka created specifically, rather than simply using Hadoop MapReduce for everything?

Best answer: Kafka was created at LinkedIn to handle high-throughput, continuously arriving event streams -- user activity events, application metrics -- which is fundamentally a different shape of problem than MapReduce's batch, finite-dataset model. Kafka focuses on reliably ingesting and delivering a never-ending stream of messages between producers and consumers, a role MapReduce was never designed to fill.

Q46. What is the practical relationship between Spark and Kafka in a modern real-time pipeline, as previewed for Unit V?

Best answer: Kafka typically acts as the ingestion and buffering layer, reliably collecting a continuous stream of events from producers and holding them until they're consumed. Spark Streaming (or Structured Streaming) then acts as the CONSUMER, reading from Kafka in small continuous batches or record-by-record, and applying processing logic -- aggregation, filtering, ML scoring -- to that stream in near real-time.

Q47. If asked to justify 'why learn Spark instead of just sticking with MapReduce,' what's the strongest, most specific answer you could give?

Best answer: The strongest answer connects the RAM-vs-disk speed hierarchy directly to a concrete performance outcome: MapReduce writes intermediate results to disk between every stage for fault tolerance, while Spark keeps intermediate results in memory as RDDs across stages whenever possible and instead achieves fault tolerance through lineage-based recomputation -- which is precisely why Spark is commonly 10 to 100 times faster than equivalent MapReduce jobs, especially for iterative workloads like machine learning training that reuse the same data across many passes.

Q48. How would you explain, to a non-technical interviewer, why 'the cloud' or 'a cluster' can process data faster than your laptop?

Best answer: A cluster isn't one computer working harder -- it's many ordinary computers each doing a small independent slice of the same job at the same time, then combining their results, the same way ten people each reading one chapter of a book finishes the whole book faster than one person reading it cover to cover alone. The engineering challenge, and what this whole course teaches, is how to split the work fairly, keep going smoothly even when one 'reader' drops out partway through, and correctly combine everyone's results at the end.

Q49. What single technical fact from this handbook would you lead with if asked 'what makes Spark fast' in an interview?

Best answer: That Spark keeps intermediate data in RAM across processing stages instead of writing it to disk after every stage the way classic MapReduce does, directly exploiting the fact that RAM access is roughly 1,000 to 100,000

times faster than disk access -- and that it still remains fault tolerant by tracking lineage rather than relying on that in-memory data being durably replicated.

Q50. Looking back across this whole handbook, what's the ONE conceptual thread that connects Linux fundamentals, distributed systems theory, and the Big Data evolution together?

Best answer: Every layer exists to manage the same underlying tension: real work needs to be spread across many unreliable, network-connected machines instead of one reliable machine, and every tool in this handbook -- environment variables that keep configuration consistent across machines, SSH that lets a master control workers, replication that survives node failure, in-memory processing that avoids the network/disk bottleneck -- is a specific engineering answer to some facet of that same core coordination-at-scale problem.

You're Ready for Unit I

At this point you can navigate a Linux filesystem, correctly set and permanently persist `JAVA_HOME/SPARK_HOME/PATH`, reason about SSH and ports well enough to debug a cluster that "won't connect," explain the JVM heap/off-heap/GC story well enough to survive later memory tuning conversations, and — most importantly — you understand WHY distributed computing exists at all: the vertical-vs-horizontal scaling ceiling, the RAM/disk/network speed hierarchy, master/worker topology, and the HDFS + MapReduce mechanics that Spark was built to improve on.

Unit I will now feel like a natural next step rather than a wall of new vocabulary: "limitations of MapReduce" is Module 3.3's disk-heavy shuffle; "batch vs. real-time"; "in-memory processing" is Module 2.2's speed hierarchy paying off; and "Spark vs. Hadoop ecosystem" is simply comparing two different answers to the same master/worker coordination problem from Module 2.3.

Before you move on

Make sure you can, without looking back at this document, explain in your own words: (1) what `PATH/JAVA_HOME/SPARK_HOME` each do, (2) the difference between vertical and horizontal scaling, (3) why RAM beats disk beats network, and (4) the three-step Map -> Shuffle -> Reduce flow. If any of these feel shaky, revisit that section's comic box and hands-on exercise before starting Unit I.